

Guida Avanzata C++: Polimorfismo e Classi Astratte

Regole d'oro per la gestione della memoria dinamica, funzioni virtuali pure e array di puntatori

Il **Polimorfismo Dinamico** permette a oggetti di classi differenti (ma derivate dallo stesso padre) di rispondere alla stessa chiamata di un metodo in modi specifici. Si realizza usando **puntatori della classe base** collegati a oggetti istanziati dinamicamente tramite la parola chiave `new`.

1. Funzioni Virtuali e la Parola Chiave `override`

Per abilitare il polimorfismo, un metodo nella classe Padre deve essere preceduto da `virtual`. Nelle classi Figlie, lo stesso metodo deve essere riscritto usando l'identificatore `override` alla fine della firma. Questo garantisce che il compilatore controlli la corrispondenza esatta dei parametri, evitando bug invisibili.

Sintassi Figlia: `void usa_mossa_speciale() override { ... }`

2. La Regola d'Oro: Il Distruttore Virtuale

Quando gestisci oggetti figli tramite puntatori della classe padre, il distruttore della classe base **deve obbligatoriamente essere virtuale**.

⚠ COSA SUCCEDDE SE TI DIMENTICHI? Se scrivi solo `~Padre()` senza `virtual`, quando eseguirai il comando `delete puntatore;` il computer eliminerà soltanto la porzione dell'oggetto appartenente al Padre. Tutti i dati esclusivi della classe Figlia rimarranno bloccati all'interno della RAM, causando un grave **Memory Leak** (perdita di memoria).

Aggiungendo `virtual ~Padre() {}`, attivi la distruzione a ritroso: viene chiamato prima il distruttore della Figlia e poi quello del Padre, pulendo la memoria al 100%.

3. Classi Astratte e Funzioni Virtuali Pure

Una classe diventa **Astratta** se contiene almeno una funzione virtuale pura, ovvero una funzione che non ha un'implementazione ed è dichiarata con la sintassi `= 0`.

```
virtual float calcola_stipendio() = 0;
```

Le classi astratte fungono esclusivamente da "progetti" o "stampi". Questo comporta che:

- **Non puoi** istanziare oggetti diretti di quella classe (es. `Dipendente d;` genererà un errore di compilazione).
- **Sei obbligato** a implementare e scrivere il codice di quella funzione all'interno di tutte le classi figlie, altrimenti anch'esse diventeranno astratte.

4. Modello di Riferimento Completo (Array di Puntatori)

Ecco lo schema riassuntivo completo per gestire collezioni di oggetti polimorfici diversi tramite un singolo array di puntatori alla classe base astratta:

```
#include <iostream>
#include <string>
using namespace std;

// Classe Base Astratta
class Veicolo {
public:
    string modello;
    Veicolo(string m) { modello = m; }

    // Distruttore Virtuale Obbligatorio
    virtual ~Veicolo() { cout << "Distruttore Padre" << endl; }

    // Funzione Virtuale Pura (Rende la classe astratta)
    virtual void mostra_tipo() = 0;
};

class Auto : public Veicolo {
public:
    Auto(string m) : Veicolo(m) {}
    void mostra_tipo() override { cout << "Auto: " << modello << endl; }
};

class Moto : public Veicolo {
public:
    Moto(string m) : Veicolo(m) {}
    void mostra_tipo() override { cout << "Moto: " << modello << endl; }
};

int main() {
    // Creazione di un Array di Puntatori alla Classe Base
    Veicolo* flotta[2];

    flotta[0] = new Auto("Audi A1");
    flotta[1] = new Moto("Aprilia SXR");

    // 1. Ciclo polimorfico di utilizzo (Uso della Freccia)
    for(int i = 0; i < 2; i++) {
        flotta[i]->mostra_tipo();
    }

    // 2. Ciclo di pulizia obbligatoria della memoria RAM
    for(int i = 0; i < 2; i++) {
        delete flotta[i];
    }

    return 0;
}
```

